

Project Deliverable #2

TUMMI

GitHub URL: github.com/veermshah/3354-TUMMI

Task Delegation

1.1 Deliverable #1

- Veer Shah:
 - GitHub Repository Setup
 - Sequence Diagram
- Joel Priyanth Kota Jona Prashanth:
 - List Assumption
 - Software Process model
- Chaiyun:
 - Design a Class Diagram to organize data for accounts, restaurants, and different food.
- Kshrugal:
 - Software Requirements
 - Case Diagram
- Owen:
 - Architectural Design
- Liya:
 - Create the Use-Case Diagram and write a few short user stories to explain how students will use the app
- Francisco:
 - Address feedback from the proposal

1.2 Deliverable #2

- Veer Shah:
 - Presentation Slides
 - Frontend Implementation
- Joel:
 - Test Plan for Software
 - Comparison to other designs
- Kshrugal:
 - Project Scheduling, Cost, Effort, and Pricing Estimation
- Chaiyun:
 - Will predict project duration and staffing estimation for the completion of the app.

- (Maybe) Implemented Code (Frontend/Backend)
- Owen:
 - Backend Implementation
- Liya:
 - Put together the Test Plan for the app – (basically a checklist of scenarios to make sure features like onboarding, filters, and favs work)
- Francisco:
 - Frontend implementation
 - Slides

Assumptions

- This project assumes that the dietary, allergies, and cultural information that the restaurants and third-party APIs provide to the TUMMI app is up to date.
- This project assumes that the users have modern smartphones with a stable internet connection and grant the app GPS location permissions. This is essential for features such as finding spots near campus.
- This project assumes that user data privacy is in compliance with the general data privacy regulations for the collection, storage, and handling of the user provided data.

Software Process Model

For the TUMMI application, our team has decided to utilize the incremental process model. This model has both elements of the linear and parallel processing flows. The incremental process model allows to produce increments of different stages of the software. This model was chosen because the TUMMI app has a clear basic set of functions and minor less important features that can be added on later. The incremental model allows us to build and deliver the basic set of features and adapt to the challenges as we face them. The first increment will focus on the core features of the application, such as user account, list of restaurants near campus, and dietary filters. The second increment can be implemented once the core features are functioning stable. This increment can include more refined features, such as personalized recommendation and initial questionnaire for preferences. The third increment will include the social features of the app, such as the trending spots near campus and allowing the users to favorite restaurants.

The incremental process model also allows us to improve with user feedback. Since the TUMMI app is completely tailored towards the users, their feedback is really important for each increment. After getting feedback of increment 1 from students, we can move forward with real world data instead of making assumptions. With this software process model, the cost of accommodating these changing customer requirements is far less compared to the waterfall model.

Addressing Feedback

Our team did not receive any specific comments or requests for changes on the project proposal, suggesting that there were no clear or obvious areas requiring revision and that most of the foundational aspects of our project were already well covered in the original submission.

Planned Actions Based on Feedback:

Although no specific revisions were suggested, we reviewed our proposal as a team to ensure it fully aligns with the objectives of our overall project. As a result, we plan to do the following:

Maintain current project direction - We will continue to develop TUMMI with the same scope described in the original proposal. Our main focus remains on creating a food recommendation app tailored for diverse student populations, especially on college campuses. We will keep our emphasis on supporting different dietary preferences such as vegan, halal, Jain, and gluten-free options, as this is a core part of our motivation.

Refine clarity and details - We plan to expand sections on how the app's filtering and recommendation logic will work. For example, we will elaborate on how the onboarding questionnaire (which asks about cravings like "soupy," "sweet," or "savory") is connected to restaurant data to create personalized suggestions.

Anticipate future improvements – Even though no changes were requested, we aim to enhance the app's overall structure by exploring additional features that complement our end goals, such as:

Integrating a campus map view to show nearby trending restaurants.

Allowing users to save favorite places and receive notifications about new menu items.

Incorporating user ratings to help others make quick decisions.

Summary:

Because no specific changes were requested, no major revisions were necessary. However, we are proactively improving depth, usability, and design clarity to better prepare for implementation in Deliverable 2.

Software Requirements

Overview: Tummi is a food recommendation app which is made for students who are looking to try new types of food or food that already matches their preferred taste. In order to get accustomed to the student's preferences, when the app is first opened the user will have to

answer a few quick questions regarding their food preferences and restrictions. Based on that information our app will recommend locations that match the user's inputted preferences.

Functional Requirements:

User account and preferences:

- The user can create/log in to an account using their email or through gmail
- The user sets their preferences by manually inputting it and answering a series of quick questions.

Onboarding questionnaire:

- The app initially asks questions about preferred cuisines, restaurants as well as dietary restrictions to learn the user's preferences.
- The information given by the user is then used to give a personalized recommendation.

Restaurant Recommendations:

- The app uses the user's current location as well as the recommendation to give a list of nearby restaurants that match.
- The recommendation is displayed by the name, cuisine type, distance as well as the rating of the restaurant.

Search and filter:

- There is a search tab/bar that can be used to display the restaurants by the name or cuisine
- Filter option can be used to filter by either rating, cuisines or distance.

Favorites and History:

- There is a favorites tab and button the user can click on when viewing restaurants to add it to their favorites list.
- The app keeps track of the recently viewed locations and searches for the user to come back to.

Non Functional Requirements:

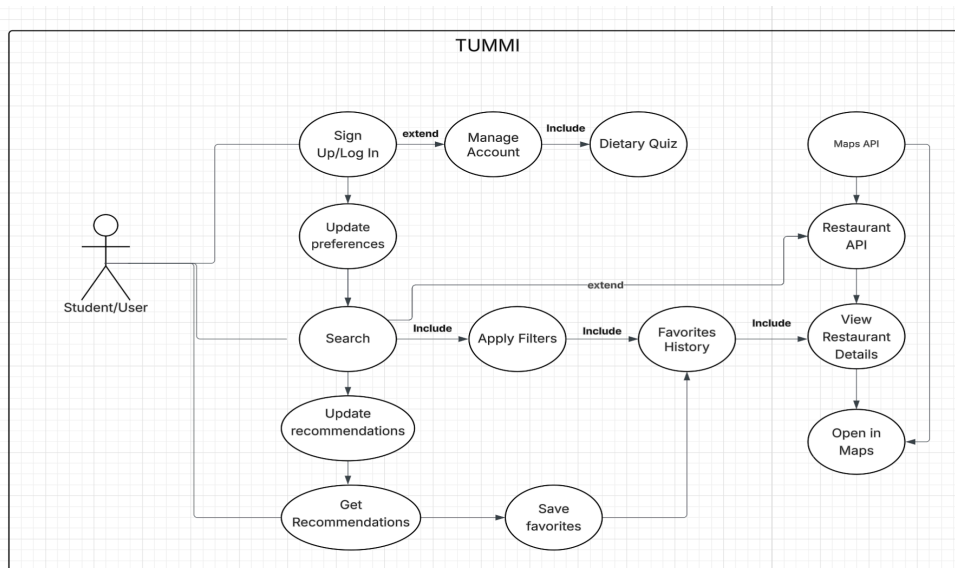
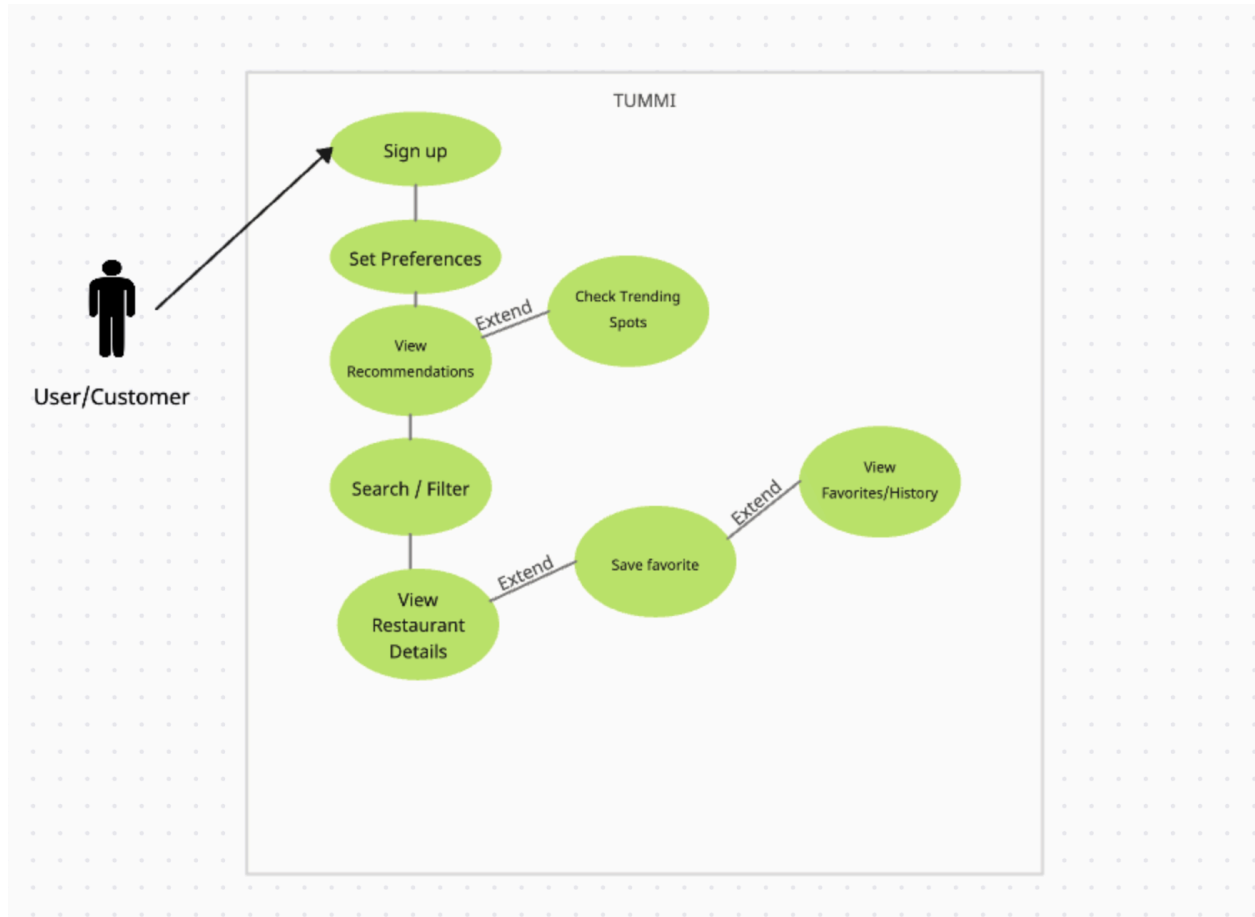
Performance: The app should be made to respond fairly quickly (for searches and recommendations) while not taking up a lot of resources.

Security: The log in info of the user should only be saved if the user ticks on the box that specifies so. None of the data of the user should be exposed.

Scalability: The app (mainly the backend) should be able to handle a lot of incoming requests by multiple accounts from the same campus.

Availability: The app should be up and available all times if it is being hosted on a cloud service.

Use Case Diagram and User Stories

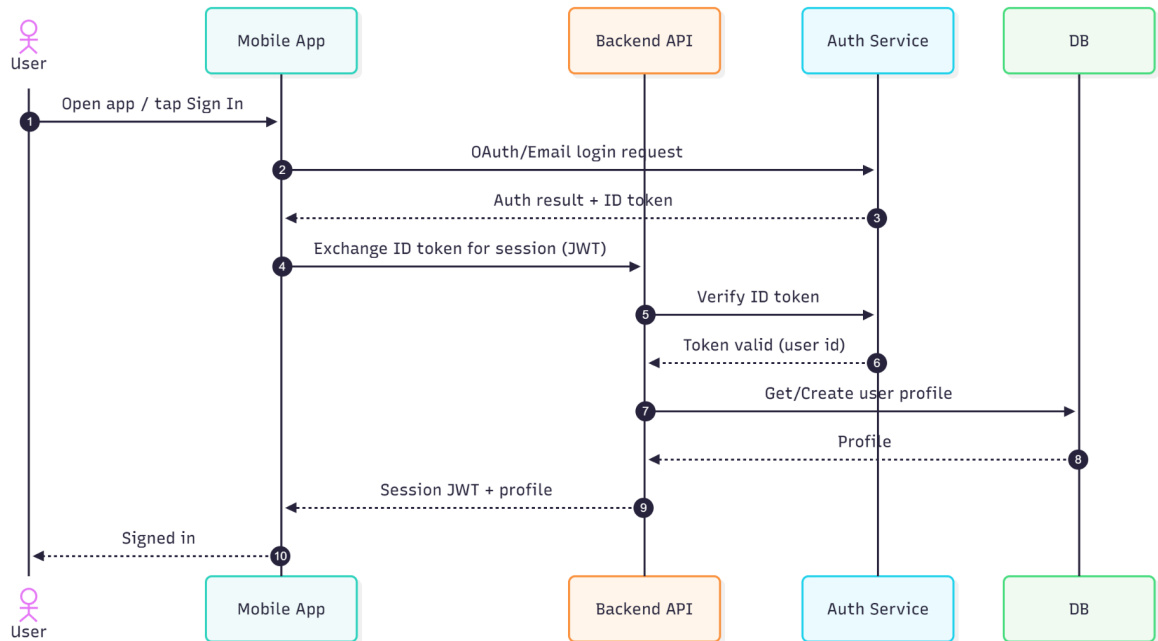


The following **user stories** summarize how a student interacts with the TUMMI app to manage their preferences and discover restaurants that fit their dietary needs:

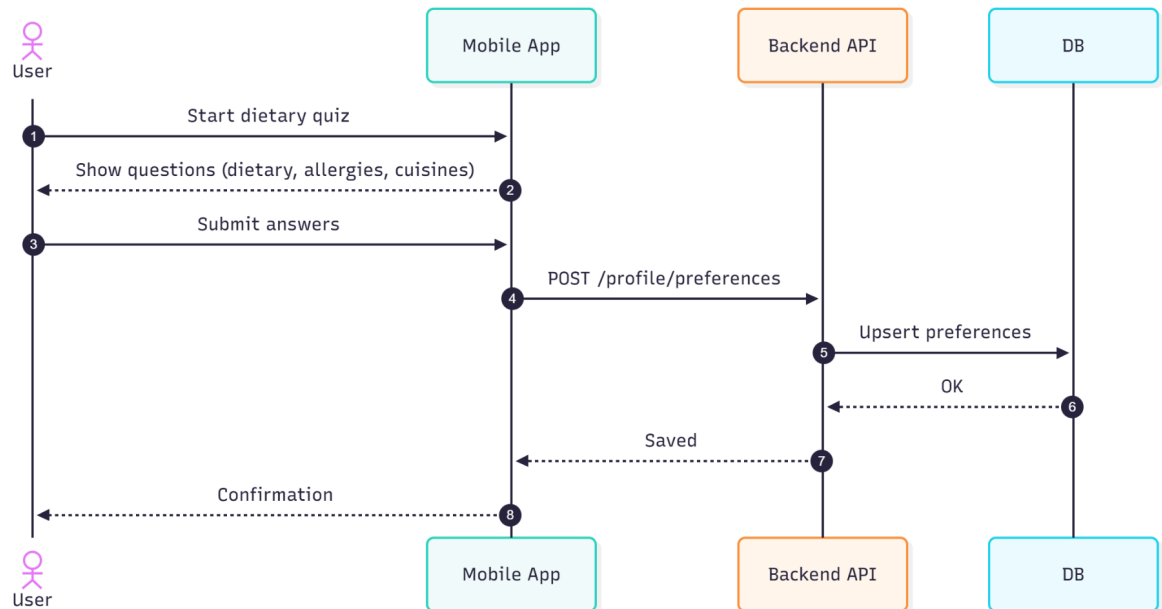
1. Sign Up / Log In: The User wants to create or access their account so they can save preferences and favorites.
2. Manage Account / Dietary Quiz: The User wants to take a dietary quiz so they can receive food recommendations that match their dietary needs.
3. Update Preferences: The User wants to update their dietary or craving preferences so their future recommendations stay accurate.
4. Search & Apply Filters: The User wants to search and apply filters (like craving, price, distance, or rating) so they can easily find a restaurant that fits what they're craving.
5. Get Recommendations: The User wants to get smart restaurant recommendations based on their saved profile and filters so they can discover new places to eat.
6. View Restaurant Details & Open in Maps: The User wants to view restaurant details and open the location in Maps so they can decide and navigate easily.
7. Save Favorites / View History: The User wants to save their favorite restaurants and view their visit history so they can revisit or keep track of places they liked.

Sequence Diagrams

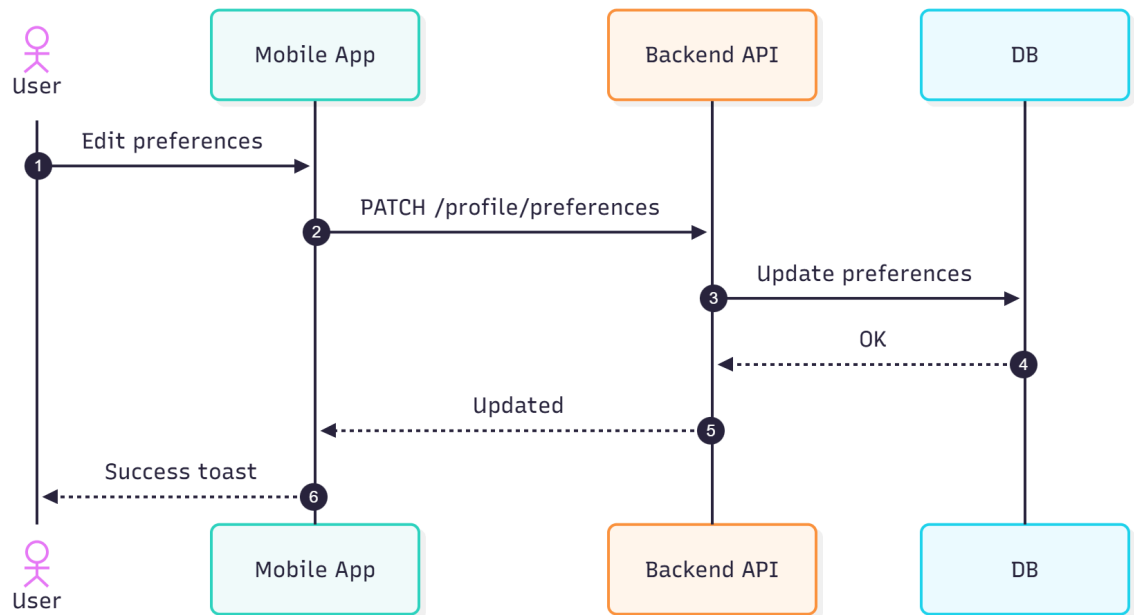
1. Sign Up/Login



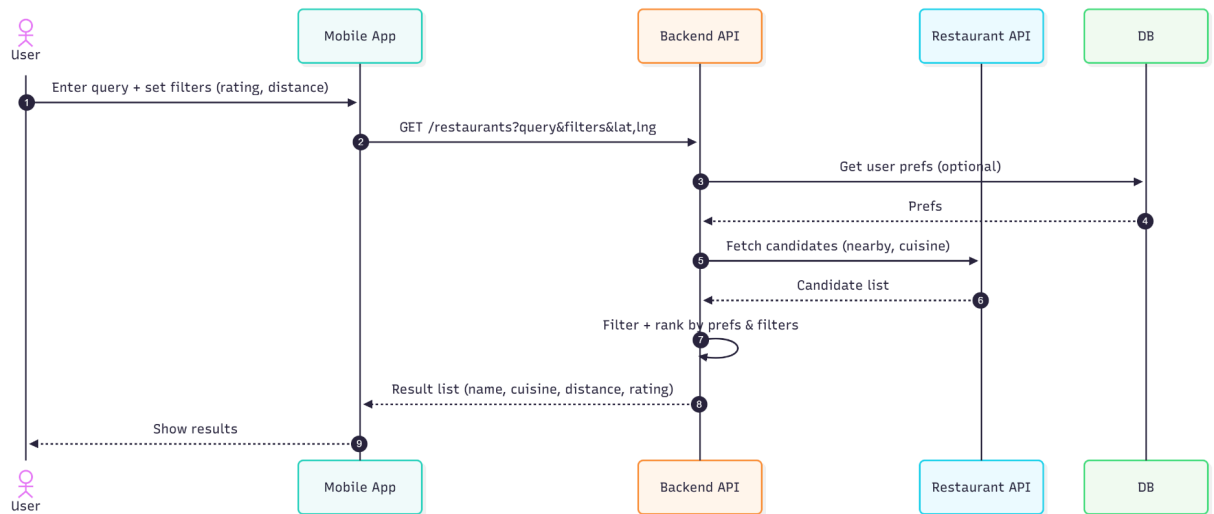
2. Manage Accounts/Dietary Quiz



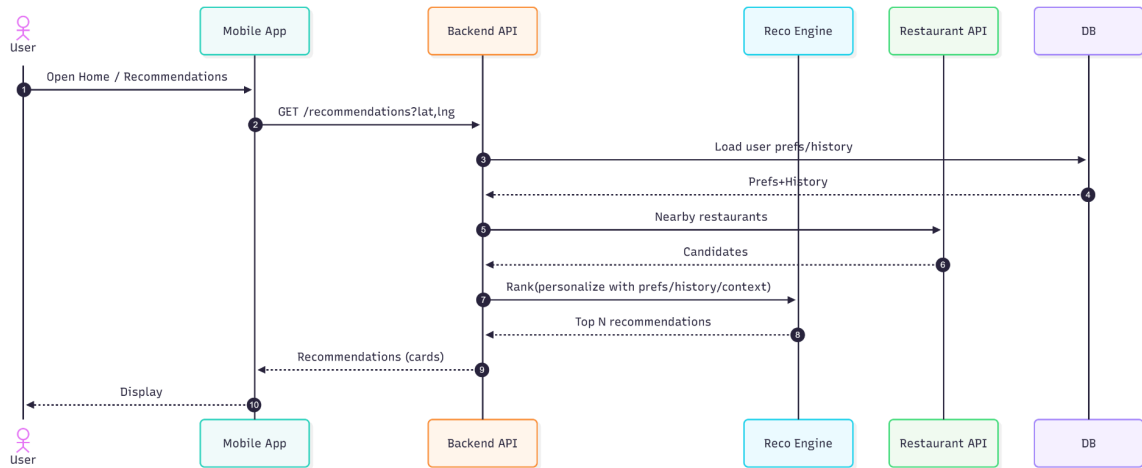
3. Update Preferences



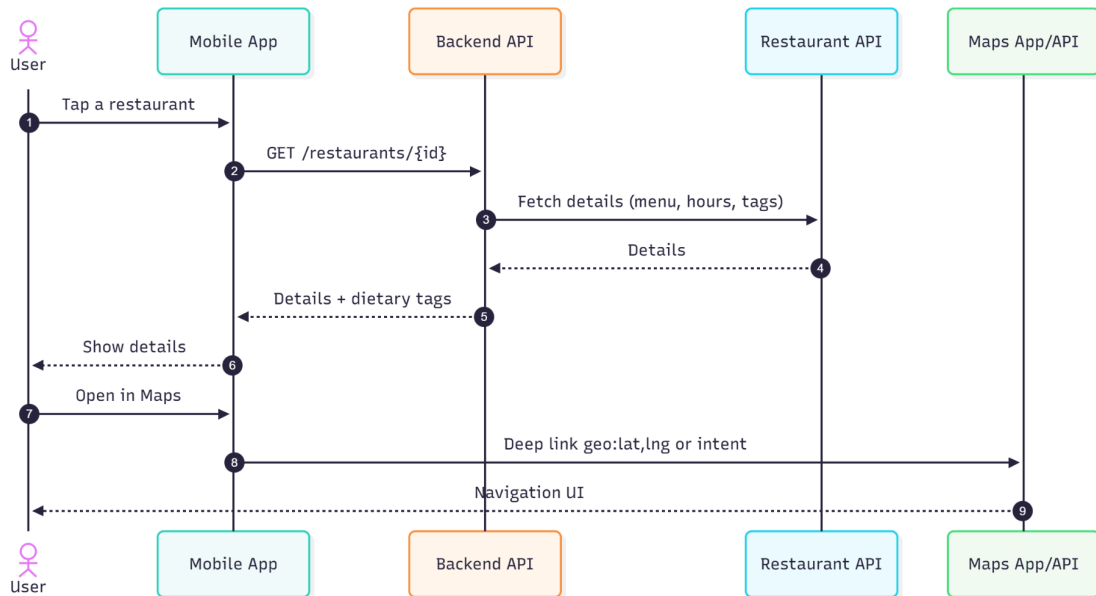
4. Search and Apply Filters



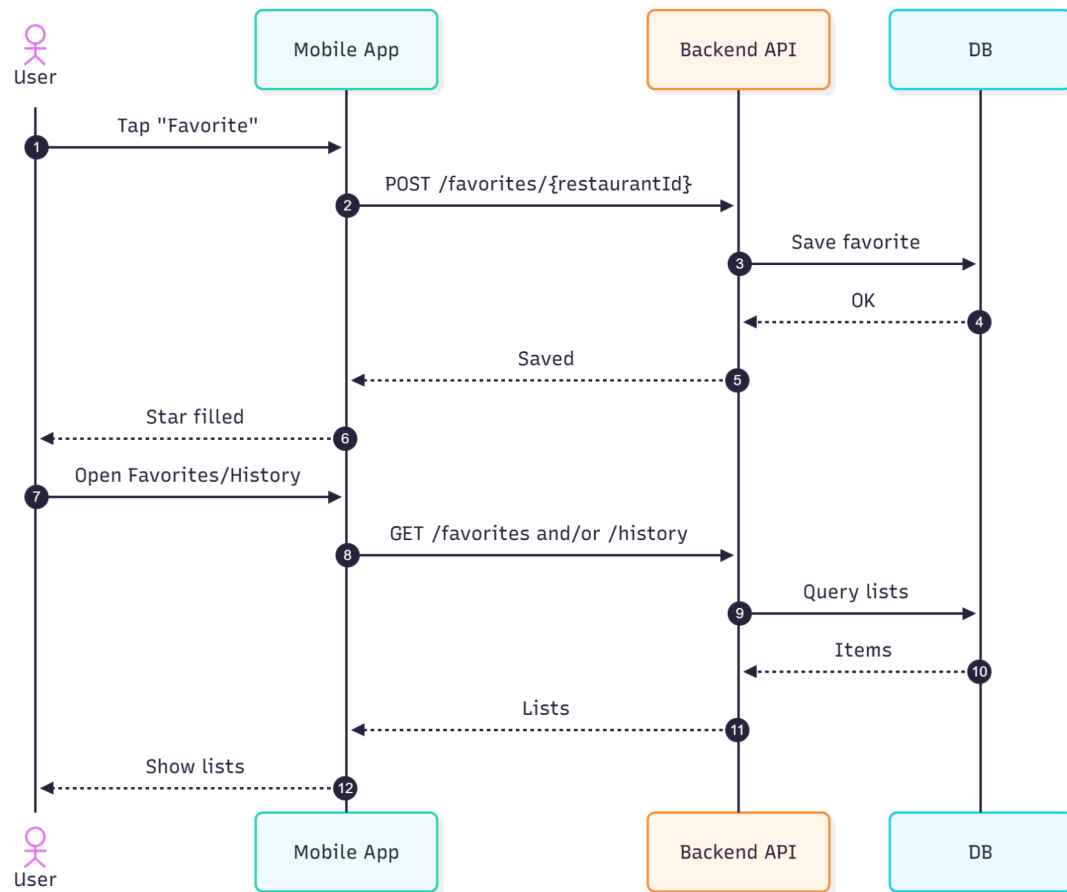
5. Get Recommendations



6. View Restaurant Details and Open Maps

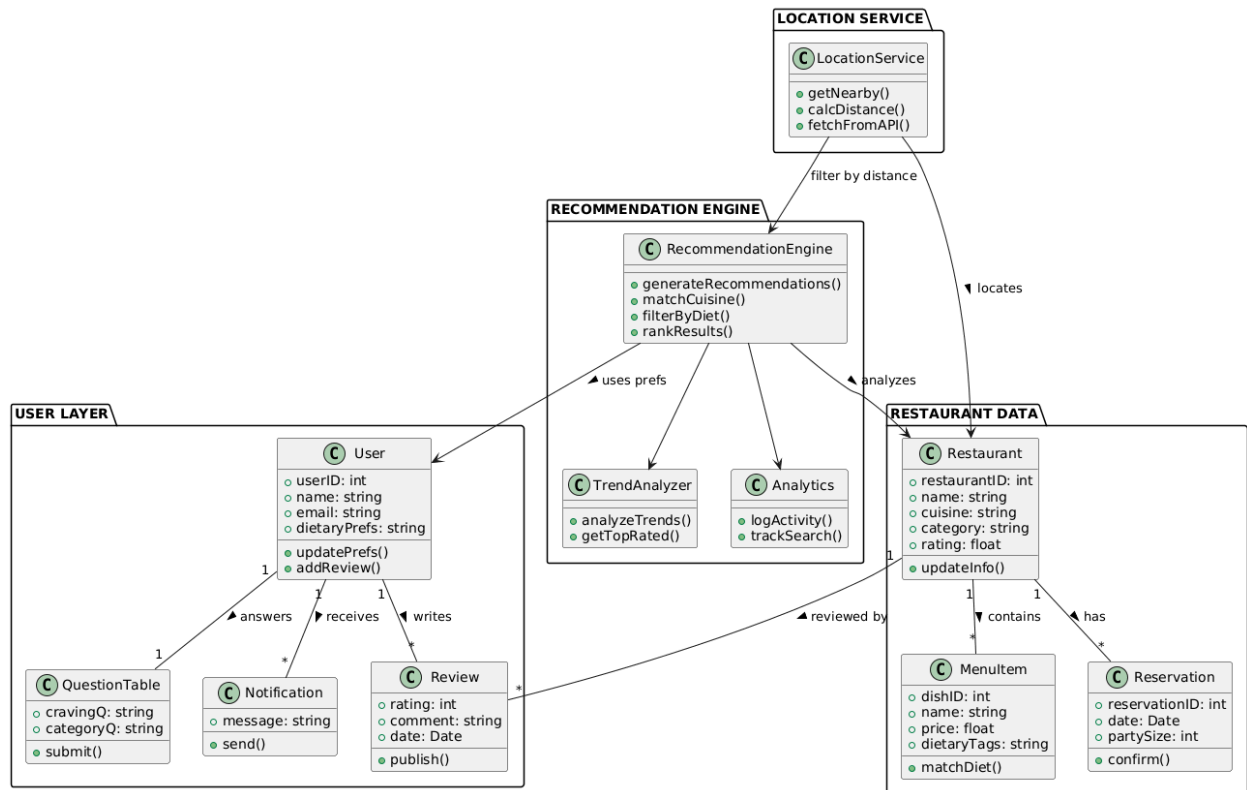


7. Save Favorites/View History

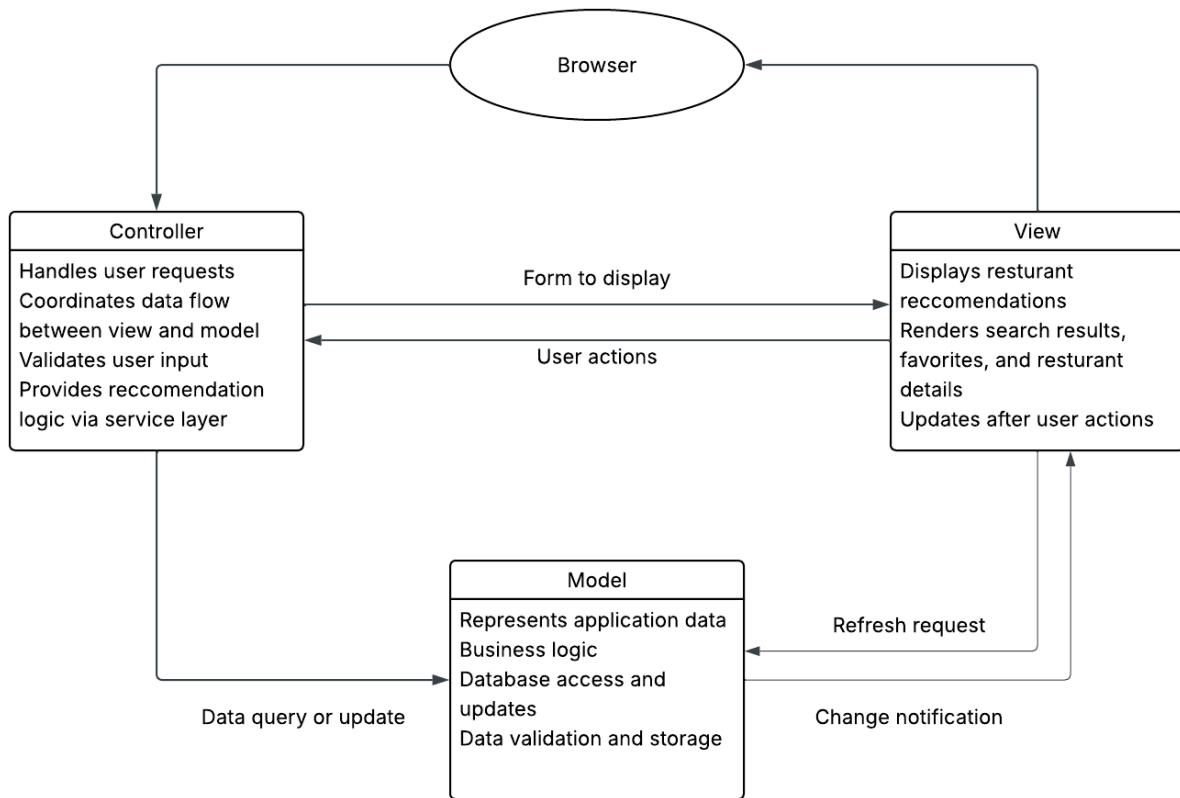


Class Diagram

TUMMI - Class Diagram



Architectural Design



Deliverable #2

Project Scheduling

Scheduling

Estimated timeline:

Start date: August 21, 2025

End date: November 23, 2025

Reason: The timeline should be within the academic semester of Fall 2025. We will be following an incremental model since we will have 3 submissions.

Submission 1: August - September (Forming the team, introductions, brainstorming and writing our proposal)

Submission 2: September - October (Our first deliverable, consisting of the requirements of our application and the various diagrams)

Submission 3: October - November (The second and final deliverable which consists of our implementation, testing, and finalization of testing for our application)

Scheduling for working:

Weekends: Weekends are excluded since we are going to be simulating a professional work week.

Working hours: We estimated that 3 hours per day per team member should be sufficient for the development of our application.

Cost, Effort, and Pricing Estimation

We will be using the Application composition model[2] to estimate the costs since that model is well versed for prototypes or early stage projects. We will be counting the number of object points (number of screens) to find the complexity.

We were able to identify 12 object points which consisted of 5 screens (Login, welcome page, home screen, search, detail page), 2 reports (history page and favorites). Using the lecture slides, the complexity weights this would total to about 12 object points.

As for productivity, since we are a team of students still learning with little experience, we will be using the productivity rating of 4 new object points a month.

This gives us a total effort of 3 months to finish the project since we have 12 object points and we will finish at a rate of 4 object points a month.

Pricing:

We will be treating this as a commercial contract. The total pricing of this project will be calculated with labor and the costs of hosting the application.

Category	Details	Cost
Personnel	3 months, 480 hours in 3 months, 21usd / hour[1]	\$10,080
Hardware	Cloud hosting service (AWS/Firebase)	\$100
Software	VS Code and Github	\$0
Total		\$10,180

Project Duration and Staffing Estimation

Estimated Duration: Approximately 13 weeks

Increment (3)	Timeline	Key Deliverables
Increment 1 – Foundation	Weeks 1-5	• UX wireframes, prototypes, and user flows • Core backend framework, database schema, & authentication • Initial restaurant listings and filtering logic • React application shell and navigation
Increment 2 – Recommendation Engine	Weeks 6-9	• Dietary questionnaire module • Preference-matching logic • Recommendation engine development • Expanded data integration • Initial QA testing

Increment 3 – Finalization	Weeks 10-13	• Trending module, favorites/history • UI refinement & Visual polishing • Full QA cycle, regression testing, performance checks • Pre-release stabilization
----------------------------	-------------	--

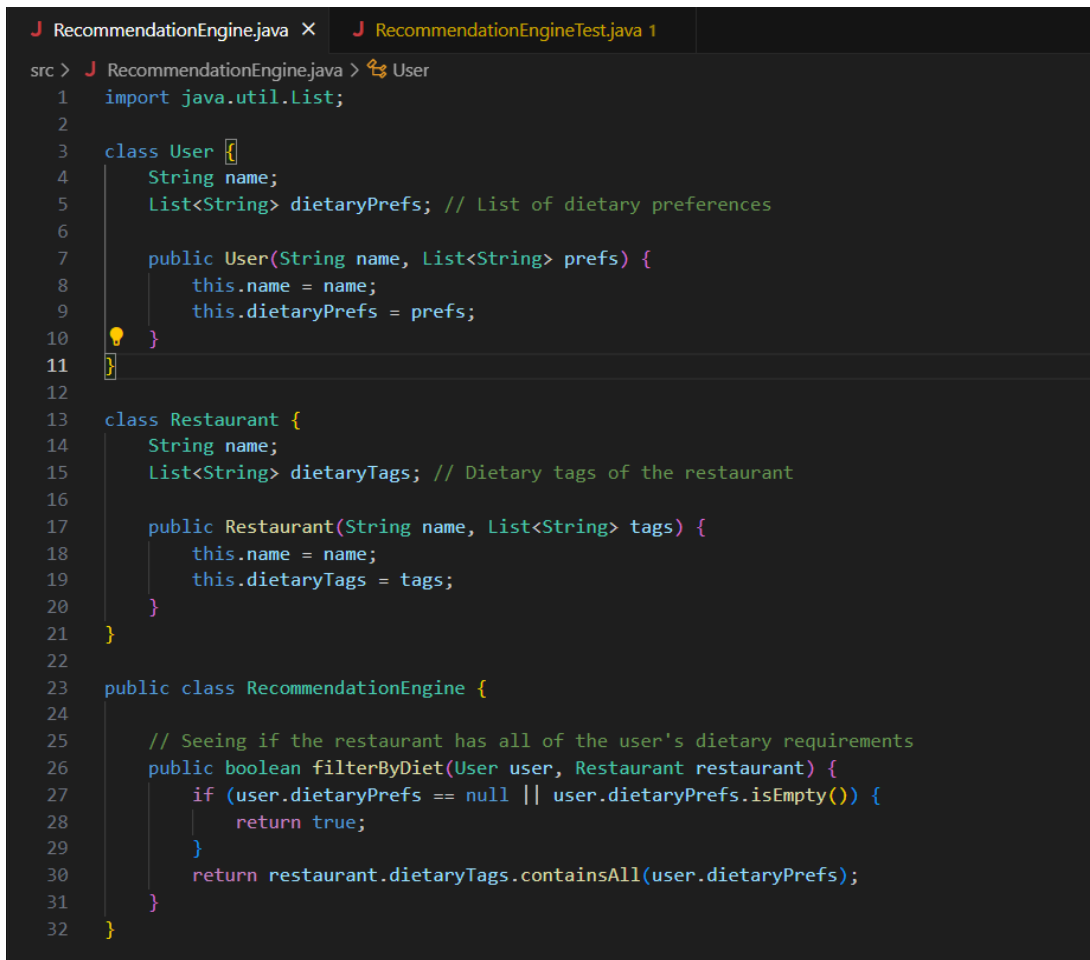
Staffing Estimation:

Role	# of staff	Hours/ Week	Duration	Total Hours	Responsibilities
UI/UX Designer	1	20	Weeks 1-4 (4)	80	Wireframes, user flows, design system, prototypes
Frontend Developer	2	40	Weeks 2-13 (12)	480 (each)	Build application interface & API hookups
Backend Developer	2	40	Weeks 2-13 (12)	480 (each)	API development, database, server logic, recommendation engine
Tester / QA	1	30	Weeks 6-13 (8)	240	Functional testing, bug tracking, & regression testing
Project Manager	1	20	Full duration (13)	260	Timeline management, sprint planning, stakeholder communication

Test Plan

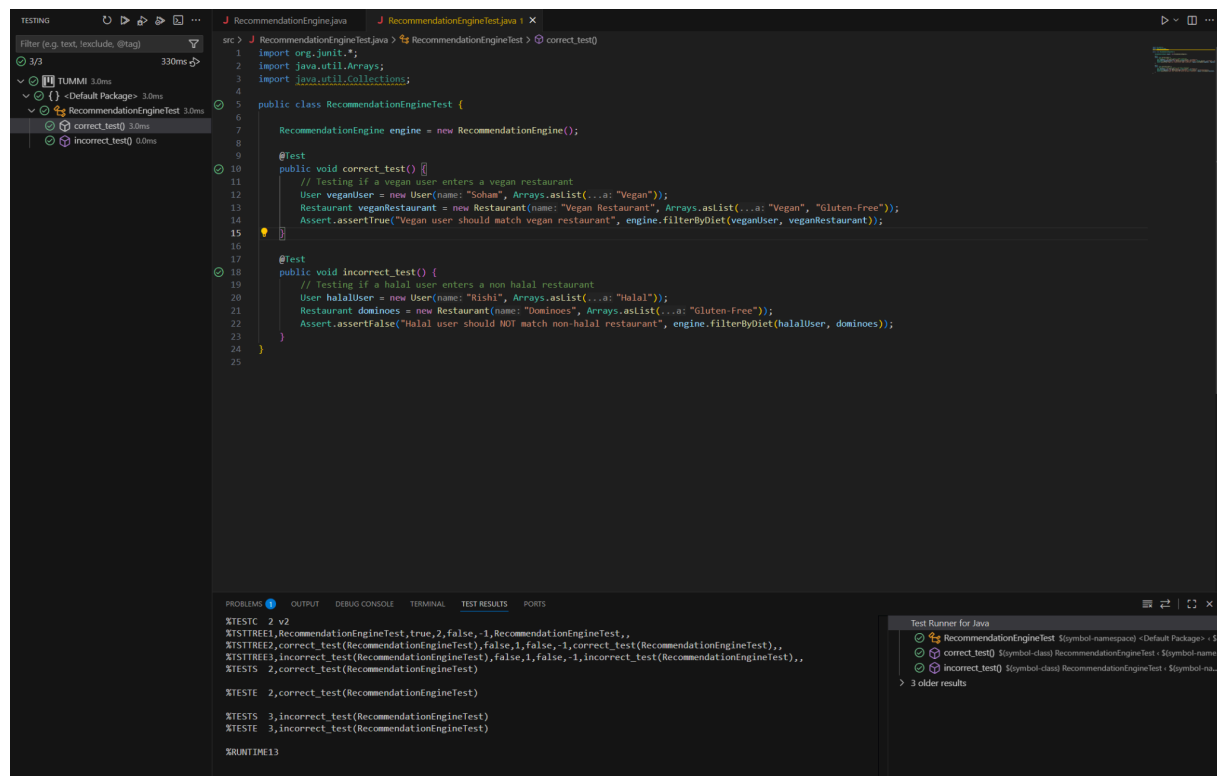
We are performing Unit Testing on one of the core features of our TUMMI application, the dietary recommendation engine. This is a fundamental proposition of TUMMI that allows students to find restaurants based on their dietary restrictions. In order for this, it is crucial that the `filterByDiet` method functions correctly. The `filterByDiet()` method under the `RecommendationEngine` class will be tested using JUnit. The test will focus on inputting user objects with different `dietaryPrefs` and verifying if the engine correctly accepts or rejects certain restaurants.

RecommendationEngine:



```
src > J RecommendationEngine.java > User
1  import java.util.List;
2
3  class User {
4      String name;
5      List<String> dietaryPrefs; // List of dietary preferences
6
7      public User(String name, List<String> prefs) {
8          this.name = name;
9          this.dietaryPrefs = prefs;
10     }
11 }
12
13 class Restaurant {
14     String name;
15     List<String> dietaryTags; // Dietary tags of the restaurant
16
17     public Restaurant(String name, List<String> tags) {
18         this.name = name;
19         this.dietaryTags = tags;
20     }
21 }
22
23 public class RecommendationEngine {
24
25     // Seeing if the restaurant has all of the user's dietary requirements
26     public boolean filterByDiet(User user, Restaurant restaurant) {
27         if (user.dietaryPrefs == null || user.dietaryPrefs.isEmpty()) {
28             return true;
29         }
30         return restaurant.dietaryTags.containsAll(user.dietaryPrefs);
31     }
32 }
```


JUnit Test and results:



For the testing requirements we defined two specific test cases in order to verify if the `filterByDiet()` method is working correctly. It was ensured that the test covered both a successful scenario and a failure scenario.

Test 1: The objective of this test was to demonstrate the system correctly identifies a restaurant as a successful recommendation when it contains the user's specified dietary restrictions. The test initialized an User object named "Soham" with a dietary restriction of "Vegan" and a Restaurant object named "Vegan Restaurant" with the dietary tags "Vegan" and "Gluten-Free". The method `engine.filterByDiet(Soham, veganRestaurant)` was called and the JUnit assertion confirmed that the boolean return value was true, validating that the system correctly identified the Vegan Restaurant as a successful recommendation for Soham.

Test 2: The objective of this test was to demonstrate the system correctly identifies a restaurant as an unsuccessful recommendation when it does not contain the user's specified dietary restrictions. The test initialized an User object named "Rishi" with a dietary restriction of "Halal" and a Restaurant object named "Domino's" with the dietary tag "Gluten-Free". The method `engine.filterByDiet(Rishi, Domino's)` was called and the JUnit assertion confirmed that the boolean return value was false, validating that the system correctly identified Domino's as an unsuccessful recommendation for Rishi.

Competitors

There are various other food discovery and dining applications that are generally used by everyone. These apps however do not properly intersect student life and dietary restrictions into the platform. These competitors excel in the broad search of restaurants and delivery logistics, they do not properly account for the decisions students face when it comes to restrictions like Halal, Jain, or severe allergies. Below is a list of three major competitors and their comparisons to TUMMI.

- Yelp / Google Maps
 - Functionality: Both yelp and google maps are industry leaders for users to discover new restaurants based on location. The platform relies on user imputed ratings and generic attributes like “Vegetarian-friendly” to categorize the restaurants.
 - Limitations: Both of these platforms assume that the user already knows what they want before searching, for example, if the user wants Thai food they will search “Thai food near me.” The dietary filters are often secondary attributes that are relied upon user reviews which can be dangerous for users with strict restrictions.
 - Our Approach: Unlike Yelp or Google Maps our initial questionnaire ensures the app captures all the dietary needs of the users immediately. The system then filters the entire database before the user sees their recommendations. TUMMI also addresses students’ decisions with craving questions, which will guide the users who do not know what they want to eat.
- Grubhub/ Uber Eats
 - Functionality: Both Grubhub and Uber Eats are widely used by university students for model ordering from on and off campus restaurants.
 - Limitations: These applications focus more on ordering ahead rather than discovering new restaurants. If partnered with the university, they are limited to on campus restaurants and rarely recommend students to explore the options in their surrounding city.
 - Our Approach: TUMMI focuses on a social exploration, instead of locking students to their on-campus food, TUMMI highlights trending spots near campus. This helps students integrate with their local community, offering a social experience that transaction apps like Grubhub and Uber Eats lack.

Conclusion

Throughout the development of TUMMI, our team successfully followed the incremental process model to deliver a well-structured and user-focused food recommendation application. Overall, our work aligned closely with the plan presented in our original proposal and Deliverable #1. The transition from planning to implementation was smooth thanks to clear task delegation, resulting in steady progress toward a functional prototype.

While our overall direction remained aligned with the proposal, a few adjustments were made during implementation. Firstly, we incorporated a more refined dietary filtering structure than we originally anticipated. Early implementation revealed that simple string-matching was insufficient for dietary needs, so we adjusted the backend model to allow restaurants to contain multiple dietary tags. The team also expanded the testing scope to include JUnit tests for the dietary recommendation engine. This is because recommendations are central to TUMMI's value, verifying correctness early helps to prevent future compounding errors and also provides increased reliability before integrating any additional recommendation layers. Some diagrams, such as the class diagram and sequence diagrams, underwent some changes thanks to some clarification after implementation began, since the real code structure helped to highlight more precise relationships between components.

Despite some minor deviations, all adjustments helped to improve our project's clarity, reliability, and overall functionality. The changes implemented were more so for refinement rather than flawed assumptions that needed correction. Thanks to our team's strong foundational planning, there were no major redirections. As a result, TUMMI has a strong technical base with core functionality that has been validated through testing, as well as clear design documentation. Our app can continue to progress post project timeline into a fully deployable application.

References

- [1] ZipRecruiter, "Computer Science Internship Salary in Texas," Nov. 2025. [Online]. Available: <https://www.ziprecruiter.com/Salaries/Computer-Science-Internship-Salary--in-Texas>. [Accessed: Nov. 22, 2025].
- [2] UTD CS3354, "Chapter 23 - Project Planning," Class Lecture Slides, University of Texas at Dallas, 2025.
- [3] UTD CS3354, "Chapter 8 - Software Testing," Class Lecture Slides, University of Texas at Dallas, 2025.

Link to Slides (also in GitHub Repo):

https://docs.google.com/presentation/d/1AW2ZigyRxhsjQIet0rs1kTTmmvp_5guqKc-Qu6uXouk/edit?usp=sharing